

Rahnuma

रहनुमा رہنما

A working guide to the ILM, PANINI and Zistgah stack, from bytes and scripts to compilers, Pāṇini and quantum substrates

प्रकृति

	written	Devanagari hub	Romenagri	and back
Devanagari	प्रकृति	→ प्रकृति	→ prak_ri_ti	→ प्रकृति
Telugu	ప్రకృతి	→ प्रकृति	→ prak_ri_ti	→ ప్రకృతి
Bengali	প্রকৃতি	→ प्रकृति	→ prak_ri_ti	→ the same identity
Urdu, as usually written	ہندوی	→ हनदवी	→ _hana_dawee	→ हनदवी
Urdu, with zer and sukun	ہندوی	→ हिन्दवी	→ _hin_dawee	→ हिन्दवी

One identity, many scripts. Telugu and Bengali meet Devanagari in one hub, become the same Romenagri word, and come back exactly. Urdu meets the hub at what is written: without its short-vowel marks the word reads hanadawee; with zer and sukun it is hindawi. Every cell here was produced by the Hindawi tools while this guide was built.

Abhishek Choudhary, AyeAI. Version 2.0.0, 3 October 2026.

Rahnuma, the edition for: School student

Contents

Part I: Orientation 2

- 1. How to use this guide 3
- 3. Start here: find your way in 4

Part II: One architecture 6

- 10. Three families, set apart: the AyeAI Triad, GramSheel and Project VIKRAM, 7
Humanesque and its projections

Part IV: Foundations 12

- 17. Numbers inside the machine 13
- 18. Characters inside the machine 15
- 19. Writing systems, sound, and reversible transliteration 18
- 20. Pāṇini's grammar as a formal system 25

Part V: Working with the estate 30

- 27. Syllabus links, and our syllabus 31

Part I: Orientation

1. How to use this guide

In plain words. This guide explains a large body of work on computers, languages and intelligence. You can read it, run the programs on your own computer, or ask an AI to help you, and every example in it was really run.

This guide is for anyone who wants to use, check or extend the work gathered in the *zistgah*, *project-ilm* and *hindawiai* organisations on GitHub: students preparing for GATE, engineers who write compilers, firmware or chip layouts, linguists who work with scripts and sound, and researchers in quantum computing and AI. It asks for curiosity and a laptop, and nothing else.

There are four ways in.

- **Understand.** Part II gives the whole architecture first: where the work comes from, how the parts compose, each term with its nearest analogue and the difference, and the three Acts.
- **Read.** Parts IV and V explain the ideas the work rests on, with the mathematics written out: how numbers and letters sit in memory, what makes a transliteration reversible, how Pāṇini's grammar computes, how a compiler turns text into machine instructions, and how an oscillator network and a quantum circuit attack the same problem.
- **Run.** Part III gives the exact commands to set up Linux, Windows with WSL, or macOS, and to run every component of the estate on your own machine.
- **Ask.** Chapter 24 shows how to put any AI assistant to work on these repositories, and how to check what it tells you.

Conventions used throughout:

- Every example that could run while this guide was being built did run, and the text shows its real output, marked *Executed while this guide was built*. The package's own `gate`, `bash ops/verify.sh`, runs those examples again and fails if any output differs from what is printed here.
- Status words are used exactly. **Tested:** a test in the repository asserts it. **Executed:** it ran and its output is shown. **Written:** the code exists but has not been run here. **Mocked:** a stand-in replaces hardware that is not attached. **Planned:** described, not built.
- Everything said about the repositories was retrieved from the repositories themselves on 29 September 2026: their listings on GitHub, their `CITATION.cff` files and their READMEs. Where a repository does not record something, this guide says so instead of filling the gap.
- A dagger, †, marks a statement that follows the author's working notes and write-ups as supplied on 29 September and 3 October 2026, rather than the repositories.
- Sanskrit is transliterated in IAST. The spelling is British.

The work lives at github.com/zistgah, github.com/project-ilm and github.com/hindawiai, on the site zistgah.org, and in the Zenodo records listed in Chapter 12.

3. Start here: find your way in

In plain words. Four steps to find your part of the guide: who you are, your board or field, what you want to do, and your path with a PDF made for you.

This guide serves a school student and a senior researcher; a village health worker and an investor. Nobody needs all of it. Four steps find your part, on the site or with this page:

1. **Who you are.** A school student, a college student, a postgraduate or doctoral researcher; a teacher; an academic researcher; a working professional, someone in public service, policy or law, someone in community, NGO or village work, an artist, writer or maker; an industry partner or investor; or an AI agent working for a person.
2. **Your board or your field.** A school student chooses a board, ICSE, ISC, CBSE, IGCSE, IB, a state board or another; and a class. Everyone else chooses a field from the international standard classification of fields of study, ISCED-F 2013, the same one FAKIR's lattice uses. It places each subject where its own scholars would: political science is under social and behavioural sciences, not the humanities; law is under business, administration and law; linguistics is under languages.
3. **What you want to do.** Learn the basics, run the software, study or research, teach, contribute, or adopt, fund or invest.
4. **Your path and your edition.** The site lists your chapters in order, with reading times and the components to try first, and offers a PDF edition made for you. Every edition below is a printed selection from this one guide, so all of them stay in step.

Edition	PDF
School student	role-school.pdf
College student	role-college.pdf
Postgraduate or doctoral researcher	role-pg.pdf
Teacher or lecturer	role-teacher.pdf
Academic researcher	role-researcher.pdf
Working professional or engineer	role-professional.pdf
Public service, policy or law	role-public.pdf
Community, NGO or village work	role-community.pdf
Artist, writer or maker	role-creator.pdf
Industry partner, investor or funder	role-industry.pdf
Generic programmes and qualifications	field-00.pdf
Education	field-01.pdf
Arts and humanities	field-02.pdf
Social sciences, journalism and information	field-03.pdf

Edition	PDF
Business, administration and law	field-04.pdf
Natural sciences, mathematics and statistics	field-05.pdf
Information and Communication Technologies (ICTs)	field-06.pdf
Engineering, manufacturing and construction	field-07.pdf
Agriculture, forestry, fisheries and veterinary	field-08.pdf
Health and welfare	field-09.pdf
Services	field-10.pdf

The first public version of this guide, written for advanced researchers and senior professionals, stays as it was at [version 1](#).

Part II: One architecture

10. Three families, set apart: the AyeAI Triad, GramSheel and Project VIKRAM, Humanesque and its projections

In plain words. Three groups explained one at a time: the AyeAI Triad of thinking, body and communication; GramSheel and Project VIKRAM for villages; and Humanesque with the names it takes in each culture.

Three families of work are easy to blur into each other or into the rest of the estate. This chapter sets each apart, with what its own record says.

The AyeAI Triad

The Triad has three members, AyeAI®, AyeAM® and AyeCNSe®, with AyeAI at the apex and AyeCNSe and AyeAM at the base. It is a triad, not a pipeline and not a loop: a loop running from the environment through AyeCNSe, AyeAI and AyeAM back to the environment was proposed elsewhere and rejected. Its formal closure gives each member a tuple:

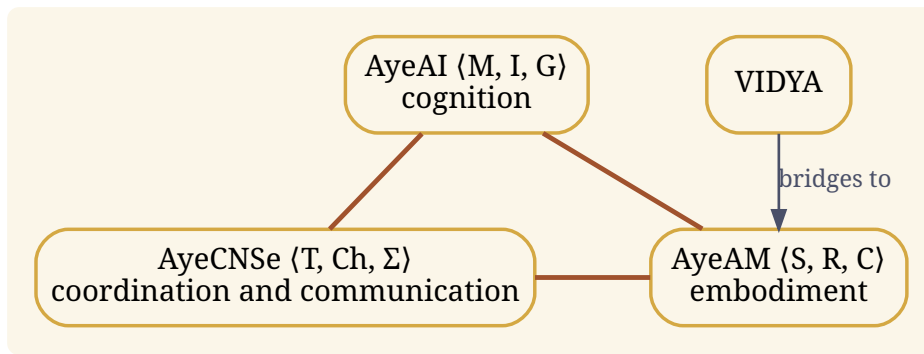


Figure 8. The AyeAI Triad: a triad with AyeAI at the apex, not a loop and not a pipeline.

$$\mathbf{AyeAM} = \langle S, R, C \rangle, \quad \mathbf{AyeAI} = \langle M, I, G \rangle, \quad \mathbf{AyeCNSe} = \langle T, Ch, \Sigma \rangle.$$

AyeAI is cognition, the apex: “the inclusive AI”, in the words of its site, and the organisation [ayei](#) holds its platforms, among them an open public stack for synthetic intelligence (opssi), virtual deployment infrastructure for scientific and cognitive computing (ayevdi), Staggered Upload™ (upload), the AyeAI Singularity Public License (spl) and the NI2A2 work.

AyeAM is embodiment: the body through which intelligence acts, across ground, air, water, space and habitat. VIDYA bridges to AyeAM without being a member of the Triad.

AyeCNSe is coordination and communication across media: the form studio at [forms.ayecnse.site](#) and the Human Context Model paper at [hcm.ayecnse.site](#) are its published faces.

Two repositories mark themselves as PEDLER and AyeAI Triad work: PRATIK’s kernel and Zamin’s ternary silicon.

GramSheel and Project VIKRAM

The GramSheel Foundation is the social architecture of the estate and the older of its lines: † it began as the Nature Welfare Society in 1993. Its principles, HEJLP, are health, education, justice, livelihood and peace. Its Village Knowledge Center (Universal Device) was a Stockholm Challenge finalist in 2010, and TWISHA runs under the same foundation.

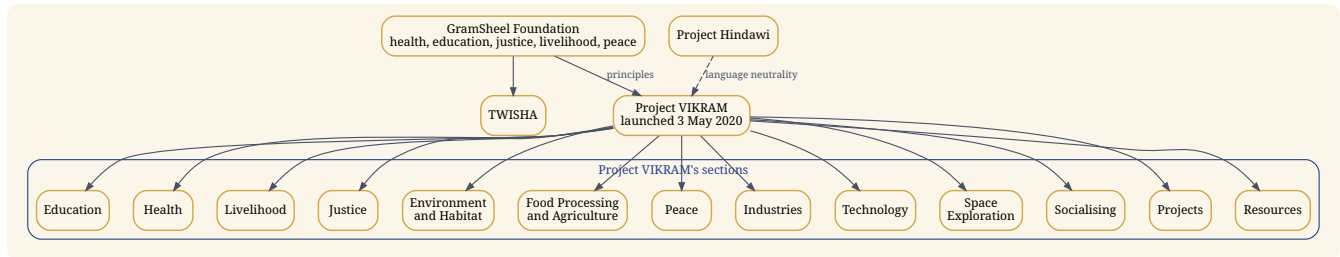


Figure 9. GramSheel and Project VIKRAM, with VIKRAM's sections as its own pages name them.

Project VIKRAM, Virtualized Infrastructure for Knowledge-driven Rural Ascension Management, is † the technical implementation of GramSheel's principles. It was launched on 3 May 2020 as the Project Vikram Journal, “an inclusive rural digitalization platform inspired by the principles of GramSheel”, under the motto गाँव बढ़ेंगे तो सब बढ़ेंगे, when the villages grow, everyone grows. Its own pages (pvjournal.github.io) name its components:

Component	What its page covers
Education	learning and schooling
Health	healthcare, open to submissions as issues, with tools for an accessible virtual hospital
Livelihood	the ISIC domains of economic activity
Justice	social justice
Environment and Habitat	the living environment
Food Processing and Agriculture	agriculture and food
Peace	harmony through social dialogue
Industries, Technology and Space Exploration	the technical frontiers it reaches towards
Socialising, Projects and Resources	the community and its shared material

With Project Hindawi, its page says, VIKRAM works towards language neutrality across technical fields. In the three Acts, GramSheel and Project VIKRAM stand on the divider between Acts II and III.

Gram Swaraj and the questions behind GramSheel and VIKRAM

Gandhi's *Hind Swaraj* (1909) argued that self-rule is more than a transfer of authority, and his *Constructive Programme: Its Meaning and Place* (1941) placed the rebuilding of daily life inside the work of independence: its eighteen items include village industries, village sanitation, basic and adult education, the provincial languages and a national language, economic equality, the kisans, labour, the adivasis and women. The estate does not present itself as a restatement of these ideas. The point of contact is a question that becomes unavoidable once computation is infrastructure: can a community govern itself if the knowledge and the machines its decisions depend on are controlled elsewhere?

On Gandhi Jayanti, 2 October 2026, the author put the questions behind GramSheel, Project VIKRAM and Humanesque in his own words. Among them:

- Can rural communities become producers of knowledge and technology, not merely consumers of systems designed elsewhere?
- Can technology distribute capability without concentrating power?
- Can artificial intelligence strengthen individual and community sovereignty rather than replace it?
- Can knowledge remain plural, multilingual and culturally grounded while becoming computationally executable?
- Can verification, provenance and human agency become constitutional properties of intelligent systems?
- Can GramSheel build the local knowledge infrastructure, can Project VIKRAM turn that infrastructure into practical capability, and can Humanesque provide a larger architecture in which intelligence, identity, agency, provenance and equity remain connected?

His answer to the old division of the world is a direction rather than a side: to march together, not as a Global South or a Global North, but towards a Global Centre, where capability is distributed and humanity remains sovereign.

Humanesque and its projections, with their names

Humanesque is the core, with its site at humanesque.site and its organisation at hmnsq: the shared executable typed hypergraph, the ontology federation, the common execution and evidence spine, and the components every projection reuses. Its constitutional ground is PoIE, from which sovereignty recurses to every level. Its core stem repositories all belong to Humanesque; each projection gives them its own culturally sensitive names and sites.

Kaivalyik, the Indic projection, including the Advaita framing (kaivalyik.org, organisation [kaivalyik](https://kaivalyik.org)): consciousness, intelligence and transcendence; it carries Synthemetic Pragmatic Realism, Kaivalyik AGI and Jyotish. **Zistgah**, زیستگاه, the Persian and Islamicate projection (zistgah.org, organisation [zistgah](https://zistgah.org)): life, planet and ecology, a habitat to live and work in; it carries the elements, the dome and the habitats on Earth (Asli Zistgah), the Moon (Zistgah-e-Mahtab) and Mars (Zistgah-e-Bahram, † conceived earlier as Mangal Kamna). **Cosmopolis**, the Greek, Western and classical projection (cosmopolis.org, organisation c-polis): societies, civilisations, space and deep time.

The stem names below are the projection-neutral names of the components. Today the working repositories live in the zistgah organisation, so most of them carry their Zistgah names; two stems already show a second projection, HPS in Hindawi and CHAKRA in Jyotish.

Stem, in Humanesque	Zistgah	Kaivalyik
The programming system	Urdu edition (urdu-ilm)	Hindawi
The observatory and its calendars	Chakra, the turning sky	Jyotish
The ground: balanced-ternary silicon	Zamin, زمین	
The water: the factory process	AAB, آب	
The air: the environmental replica	Fiza, فضا	
The shared virtual dome	the dome	
The Cognitive Workbench	Mez, میز	
The press	Matba, مطبع	
The cutting room, the visual cyler	Khwab, خواب	
The listening room, the audio cyler	Awaz, آواز	
The immersive cyler	Tilasm, طلسم	
The embodied cyler	Pench	
The record	Yadein, یادیں	
The book template	Kitab, کتاب	
The domain spine	Dhancha, ڈھانچہ	
Communication engineering	Ertabat, ارتباط	
Device control	Zasab, اعصاب	
The door for any AI	Darwaza, دروازہ	
Scan and store	Tasvir, تصویر	
The plates	Lawh, لوح	
The parser federation	Tajziya, تجزیہ	
The guide	Rahnuma, رہنما	
The Geometry of Becoming	Tabdili, تبدیلی	
The Geometry of Living	Zindagi, زندگی	
The Geometry of Persisting	Qaem, قائم	
Topological Bildung	Tarbiyat, تربیت	

Some stems keep one name in every projection: Humanesque, ILM, Romenagri, PANINI with PANINIq, PANINIb and PANINIphy, PEDLER, QEDLER, PRATIK, CEM, UKOP, FAKIR, GENIE, TransEg, PoIE, VGC, COPA, and the provenance systems Tok DOI, spiguard, Candor and Misty DOI. Kaivalyik's and Cosmopolis's own names for the rest are given with their organisations, as each projection takes its repositories.

Part IV: Foundations

17. Numbers inside the machine

In plain words. How a computer stores numbers using only ones and zeros, including negative numbers and fractions.

A processor holds voltages that stand for bits. Everything else on this page, including the hexadecimal digits, is notation for people.

Positional notation

A number written with digits d_i in base b is

$$n = \sum_{i=0}^{k-1} d_i b^i, \quad 0 \leq d_i < b.$$

So $42 = 101010_2 = 2A_{16}$: the letters A to F are simply names for the values ten to fifteen, chosen so that one hexadecimal digit covers exactly four bits. The machine never sees the letter A, any more than it sees the letters of a keyword in a program.

Signed integers: two's complement

With n bits $b_{n-1} \dots b_0$, the value is

$$v = -b_{n-1} 2^{n-1} + \sum_{i=0}^{n-2} b_i 2^i, \quad -2^{n-1} \leq v \leq 2^{n-1} - 1.$$

Negative x is therefore stored as $2^n - |x|$. The payoff is that one adder serves signed and unsigned arithmetic alike, which is why every processor in use works this way.

Reals: IEEE 754

A double-precision number has a sign bit s , eleven exponent bits e and fifty-two fraction bits f , and for normal numbers

$$v = (-1)^s \times (1.f)_2 \times 2^{e-1023}.$$

One tenth has no finite binary expansion, so it is stored rounded, and sums of rounded values are rounded again.

Byte order and bit order

A 32-bit value occupies four bytes. A little-endian machine (x86, and ARM as usually configured) stores the least significant byte at the lowest address; a big-endian machine stores the most significant byte first, and network protocols use big-endian, which is why C has `htonl` and `ntohl`. That is endianness, and it concerns bytes.

Bit numbering is a separate convention: whether bit 0 is the least or the most significant bit. The hexadecimal digit E is **1110** when written most significant bit first and **0111** when written least significant bit first, and a serial line such as a UART sends the least significant bit first. Both conventions are representations. The circuit has only the wires.

Executed while this guide was built: `python3 examples/numbers_demo.py`

Integers and reals

```
0x0A0B0C0D little-endian bytes: 0d 0c 0b 0a    big-endian: 0a 0b 0c 0d
-5 in 8-bit two's complement: 11111011    (256 - 5 = 251)
0.1 as an IEEE 754 double: 3fb999999999999a = 0x1.999999999999ap-4
0.1 + 0.2 == 0.3 is False; the sum is 0.30000000000000004
```

These are the facts the GATE syllabus asks for under number representation and computer arithmetic, fixed and floating point (Chapter 26).

18. Characters inside the machine

In plain words. How letters from any script become numbers inside a computer, and why a name must survive every tool a program passes through.

ASCII

Seven bits, 128 codes: 0 to 31 and 127 are control codes, 32 to 126 are printable, and **A** is 65, or 0x41. It was designed in the 1960s for English teleprinters, and nearly every later character code keeps it as its first 128 positions.

ISCII

ISCII, the Indian Script Code for Information Interchange (IS 13194:1991), is an 8-bit code. Codes 0 to 127 are ASCII. The upper half holds one layout shared by the Brahmi-derived scripts of India: the same code means KA in Devanagari, Bengali, Gurmukhi, Gujarati, Oriya, Tamil, Telugu, Kannada and Malayalam, and an attribute code selects the script for display. The shared layout records the common phonological organisation of these scripts. The price is that the script, and with it the language, has to be known from context.

Unicode

Unicode has room for 1,114,112 code points, U+0000 to U+10FFFF, in 17 planes of 65,536. Plane 0, the Basic Multilingual Plane, holds every major Indic block. Each script has its own block, so KA has a different code point in each. The blocks were laid out from ISCII's order, however, so KA sits at offset 0x15 and the virama at offset 0x4D in every one of the nine: the commonality ISCII carried in a single code, Unicode carries as a fixed offset.

UTF-8

A code point U is written in one to four bytes:

$$\begin{aligned} U < 2^7 : & \quad 0xxxxxxx \\ U < 2^{11} : & \quad 110xxxxx \ 10xxxxxx \\ U < 2^{16} : & \quad 1110xxxx \ 10xxxxxx \ 10xxxxxx \\ U < 2^{21} : & \quad 11110xxx \ 10xxxxxx \ 10xxxxxx \ 10xxxxxx \end{aligned}$$

The bits of U fill the **x** positions from the left. Every Devanagari code point lies between 2^{11} and 2^{16} , so it takes three bytes in UTF-8 against two in UTF-16. Because no continuation byte (**10xxxxxx**) can be mistaken for a leading byte, a reader can find character boundaries from any point in a stream, and an ASCII byte never appears inside a multi-byte character, which is why UTF-8 passes safely through tools written for ASCII.

Executed while this guide was built: `python3 examples/chars_demo.py`

1. The same letter KA in nine Unicode blocks: every block keeps ISCII's order, offset 0x15

Devanagari	block U+0900	KA U+0915 क	virama U+094D
Bengali	block U+0980	KA U+0995 ঙ	virama U+09CD
Gurmukhi	block U+0A00	KA U+0A15 ਙ	virama U+0A4D
Gujarati	block U+0A80	KA U+0A95 ૙	virama U+0ACD
Oriya	block U+0B00	KA U+0B15 ଙ	virama U+0B4D
Tamil	block U+0B80	KA U+0B95 ங	virama U+0BCD
Telugu	block U+0C00	KA U+0C15 ఙ	virama U+0C4D
Kannada	block U+0C80	KA U+0C95 ೙	virama U+0CCD
Malayalam	block U+0D00	KA U+0D15 ങ	virama U+0D4D

2. UTF-8 by hand for U+0915 (KA): 16 bits split 4 + 6 + 6 into 1110xxxx 10xxxxxx 10xxxxxx
0000 100100 010101 -> 11100000 10100100 10010101 = E0 A4 95
Python agrees: E0 A4 95
the Devanagari block U+0900 to U+097F is E0 A4 80 to E0 A5 BF

3. Words are sequences of code points; aksharas are clusters of them
प्रकृति: 7 code points, 21 UTF-8 bytes: U+092A U+094D U+0930 U+0915 U+0943 U+0924 U+093F
पितृ: 4 code points, 12 UTF-8 bytes: U+092A U+093F U+0924 U+0943
क्षि: 4 code points, 12 UTF-8 bytes: U+0915 U+094D U+0937 U+093F
categories in क्षि: U+0915 Lo, U+094D Mn, U+0937 Lo, U+093F Mc
4. Normalisation: QA U+0958 decomposes, and composition does not rebuild it
NFD(U+0958) = U+0915 U+093C
NFC(that) = U+0915 U+093C
equal as strings before normalising: False; after NFC on both: True

What the output shows

Clusters. A reader sees क्षि as one unit, a grapheme cluster in the terms of Unicode's UAX #29, but it is four code points: KA (a letter, category Lo), VIRAMA (a nonspacing mark, Mn), SSA (Lo) and VOWEL SIGN I (a spacing mark, Mc). The vowel sign is stored after the consonant and drawn before it. Reordering and conjunct formation happen when text is rendered, in a shaping engine such as HarfBuzz working from the font's OpenType tables. Storage order is phonetic; display order is visual. Cursor movement, deletion, search and string length therefore have to work on clusters, not on code points.

Normalisation. QA has its own code point, U+0958, and also a decomposed spelling, KA followed by NUKTA (U+0915 U+093C). The two look identical and compare unequal until normalised, and because U+0958 is a composition exclusion, normalising to NFC yields the decomposed form. A lexer that accepts identifiers in Indic scripts has to normalise them, or two names that look the same will be different names.

Joiners. ZERO WIDTH JOINER and ZERO WIDTH NON-JOINER (U+200D, U+200C) ask for, or prevent, a joined form: KA, VIRAMA, ZWJ, SSA requests a half-form of KA, and KA, VIRAMA, ZWNJ, SSA requests a visible virama. They are invisible, and invisible characters are a classic route for impersonation.

Confusables. Characters from different scripts that look alike, such as Latin **a** and Cyrillic **а**, let one name pose as another. Unicode's UTS #39 defines how to detect confusable strings, with its data in `confusables.txt`, and sets rules for mixing scripts in one identifier. Any system that accepts identifiers in several scripts should apply them.

A name has to survive the whole toolchain

Python 3 accepts identifiers in any script under PEP 3131, and GCC has accepted UTF-8 identifiers in C since version 10. That settles only the first step, and for systems work it settles the wrong one.

A keyword disappears during compilation. By the time a program is machine code, a loop written with `for` and the same loop written with `क्रम` have both become a comparison and a branch (Chapter 21). A name does not disappear. Every function, global variable, parameter and register mapping lives on: in the object file's symbol table, `.symtab` with its strings in `.strtab`; in the DWARF debugging information; in linker scripts and map files; in the kernel's own symbol table, from which a panic trace is printed; and in every tool a developer points at the binary: `nm`, `objdump`, `addr2line`, GDB, OpenOCD driving a JTAG or SWD probe on a microcontroller; a vendor's IDE. Each of these has its own rules about which bytes a symbol may contain, and the one alphabet every one of them accepts is letters, digits and the underscore.

So the question for firmware, kernels and microcontrollers is not whether a compiler accepts a name written in Devanagari. It is whether that name comes back, byte for byte, at every one of those tools, and can still be read against the source. Hindawi's answer, since 2003, is Romenagri: every name travels as a word over A to Z, a to z and the underscore, which every tool in the chain accepts, and the mapping is bijective, so the name in the script can be recovered from the binary at any point. Chapter 19 sets out the scheme and Chapter 21 runs it through GCC, the ELF symbol table, DWARF and GDB.

Composing conjuncts

Hindawi's APCISR composed half-consonant conjuncts of any length in software, in the years when display hardware for Indian scripts had fixed limits on them. Shaping engines solve the same problem today. It still matters wherever text must be shown without a full shaping stack, for example on a boot console before an operating system has loaded its fonts.

19. Writing systems, sound, and reversible transliteration

In plain words. How Romenagri turns Indian scripts into plain English letters and back again without losing anything, and why Urdu written without its vowel marks is harder

Kinds of writing system

- An **alphabet** writes consonants and vowels as separate letters of equal standing: Latin, Greek, Cyrillic.
- An **abjad** writes consonants, with vowels left out or added as optional marks: Arabic, Hebrew, and the Perso-Arabic scripts of Urdu, Sindhi, Kashmiri and Punjabi in Shahmukhi.
- An **abugida** writes a consonant with an inherent vowel; other vowels are marks on the consonant, and a virama removes the vowel: Brahmi and all its descendants, Devanagari, Bengali, Gurmukhi, Gujarati, Odia, Tamil, Telugu, Kannada, Malayalam and more, and also Ethiopic.
- A **syllabary** has one sign per syllable: Japanese kana.
- A **logography** writes words and morphemes: Chinese characters. Egyptian hieroglyphs mix logograms with signs for sounds.

The Brahmi abugidas record the phonology of their languages closely, place of articulation by place of articulation, and that is why they can carry a whole computing stack: a mapping between their letters and a small ASCII alphabet can be made reversible, and the reversibility can be proved (below). In this work, full reversibility has so far been reached for the Brahmi family; for Latin written with diacritics, and for abjads that leave vowels unwritten, it has not yet been reached.

Script, language, standard: the three axes

ILM keeps three things apart.

Axis	What it is	Example
Script	How the text is written; handled by transliteration tables	Punjabi in Gurmukhi and in Shahmukhi: one language, two scripts
Language	The words: the vocabulary of keywords and names	Devanagari for Hindi, Marathi, Nepali and Sanskrit: one script, four languages
Standard	The computational construct and its host realisation	the C11 and C17 standards; a <code>for</code> loop in C, Python and Rust

A programming system that confuses these axes cannot grow past its first language. The pipeline in Chapter 21 applies them in order: script first, then language, then standard.

One spelling, several pronunciations

The consonant letter carries an inherent vowel, but languages pronounce it differently and drop it in different places.

Language	Inherent vowel	कमल (lotus) is said	Notes
Hindi, as spoken in Uttar Pradesh	a schwa, [ə]	kamal	the final inherent vowel is deleted, and some medial ones
Marathi	a schwa, [ə]	kamal	final deletion, as in Hindi
Bengali	an open o, [ɔ], raised to [o] before a high vowel in the next syllable	komol	the inherent vowel is usually kept inside a word and dropped at its end
Kannada	[a]	kamala	every vowel is pronounced; no deletion

The vowel written क, ṛ in IAST, is a second example. Hindi as spoken in Uttar Pradesh says it as *ri*; Marathi, Gujarati, Kannada and Telugu say it as *ru*. So प्रकृति is *prakṛiti* in Hindi and *prakṛuti* in Marathi and Kannada, and Bengali, which also says *ri* but rounds the inherent vowel, has *prokṛiti*. The three spellings pitri, pitru and pitra seen for पितृ record three spoken realisations of the same written vowel.

The consequence is exact: a Unicode string records the spelling, not the sound. A text-to-speech system, a transliterator or a keyword table has to know the language, not only the script, and the same Devanagari text read in Lucknow and in Pune is a different sequence of sounds.

Transliteration, transcription and romanisation

Transliteration maps letters to letters and can be made reversible. **Transcription** maps sounds to symbols, usually IPA, and is not reversible to a spelling. **Romanisation** is any rendering in Latin letters. IAST and ISO 15919 transliterate with diacritics (ṛ, ṣ, ṇ); Harvard-Kyoto and ITRANS use plain ASCII with capitals and letter pairs.

Romenagri

The Romenagri Transliteration System has been written since 2003 and released under the GNU GPL. Its current tree is the `Romenagri` folder of [hindawiai/chintamani](#), with a Perso-Arabic seed library in [project-ilm/romenagri](#). It is ASCII-7 and uses no diacritics. Its alphabet is exactly the characters a C identifier may contain apart from digits: A to Z, a to z and the underscore. The underscore is its one marker. It prefixes the independent vowels (अ is `_a`, आ is `_aa`); it separates the dental stops from the retroflex ones (त is `_ta`, ट is `ta`); it marks ह, the anusvara and the visarga (`_ha`, `ka_m`, `ka_hh`); and it keeps apart the readings that a plain Latin spelling would merge: ख is `kha`, क followed by ह is `ka_ha`, and क joined to ह by a virama is `k_ha`. A consonant written with no vowel after it joins the next consonant, as the virama does: क्ष is `kx`, and प्र begins `pr`.

Hindawi uses it through four filters. `uni2acii` turns Unicode into ACII, the 8-bit code, laid out after ISCII, in which the tools work; `acii2cf` writes ACII in Romenagri's compiler form, in which every word in the script becomes its Romenagri word while English words and the contents of string literals pass through untouched; `rmn2acii` and `acii2uni` go back. The output below was produced by those filters, built from the retrieved sources by `examples/hindawi.sh`, not typed in, and every entry was converted back and compared.

Executed while this guide was built: `bash examples/hindawi.sh words`

Letters, as Devanagari = Romenagri:

अ = _a	आ = _aa	इ = _i	ई = _ee	उ = _u	ऊ = _oo
ऋ = _ri	ए = _aee	ऐ = _ai	ओ = _oa	औ = _ou	क = ka
ख = kha	ग = ga	घ = gha	ङ = _nga	च = cha	छ = chha
ज = ja	झ = jha	ञ = _yna	ट = ta	ठ = tha	ड = da
ढ = dha	ण = _nna	त = _ta	थ = _tha	द = _da	ध = _dha
न = na	प = pa	फ = pha	ब = ba	भ = bha	म = ma
य = ya	र = ra	ल = la	व = wa	श = sha	ष = xa
स = sa	ह = _ha	ळ = lvra	क्र = kza	ज़ = jza	फ़ = phza
ड़ = _rda	ढ़ = _rdha	कं = ka_m	कः = ka_hh		

Names, to Romenagri and back through the inverse:

योग -> yoaga -> योग
जोड़ी -> joa_rdoa -> जोड़ी
सीमा -> seemaa -> सीमा
गिनती -> gina_tee -> गिनती
प्रकृति -> prak_ri_ti -> प्रकृति
पितृ -> pi_t_ri -> पितृ
क्षि -> kxi -> क्षि
ज्ञान -> j_ynaana -> ज्ञान
हृदय -> _h_ri_daya -> हृदय
उत्तर -> _u_t_tara -> उत्तर
क्षेत्रफल -> kxae_e_traphala -> क्षेत्रफल
त्रिकोण -> _trikoa_nna -> त्रिकोण
ख -> kha -> ख
कह -> ka_ha -> कह
कह -> k_ha -> कह

67 of 67 come back identical; every Romenagri form uses only A-Z, a-z and _: yes

ISO 15919, the international standard for transliterating Indic scripts, has a 7-bit fallback of its own, and Romenagri's tree records the correspondence in `iso15919_map.csv`. That fallback spells with punctuation:

Letter	ISO 15919	ISO 15919, 7-bit	Romenagri
ट	ṭ	.t	ta
त	t	t	_ta
ण	ṇ	.n	_nna
ञ	ṇ̐	~n	_yna
श	ś	;s	sha
ष	ṣ	.s	xa
ऋ	ṛ	,r	_ri

The first three columns are from `iso15919_map.csv`; the Romenagri column is the filters' output above. A full stop, a semicolon, a comma or a tilde cannot appear in a C identifier, an assembler symbol or an unquoted linker-script name, so the standard's own ASCII form cannot carry a program's names through a toolchain. Romenagri can, and that is why it exists.

One identity across scripts

ISCII gave every Brahmi script one layout, and Hindawi builds on that. Its `flatten_uni_dev`, generated from `iscii_map.csv`, maps each letter of Bengali, Assamese, Gurmukhi, Gujarati, Oriya, Tamil, Telugu, Kannada and Malayalam to its Devanagari counterpart. Devanagari is the hub; Romenagri runs on the hub; filters such as `fltr_hi_te` carry the hub back out to a script. The same word written in three scripts therefore has one identity, `prak_ri_ti`, which the cover of this guide shows.

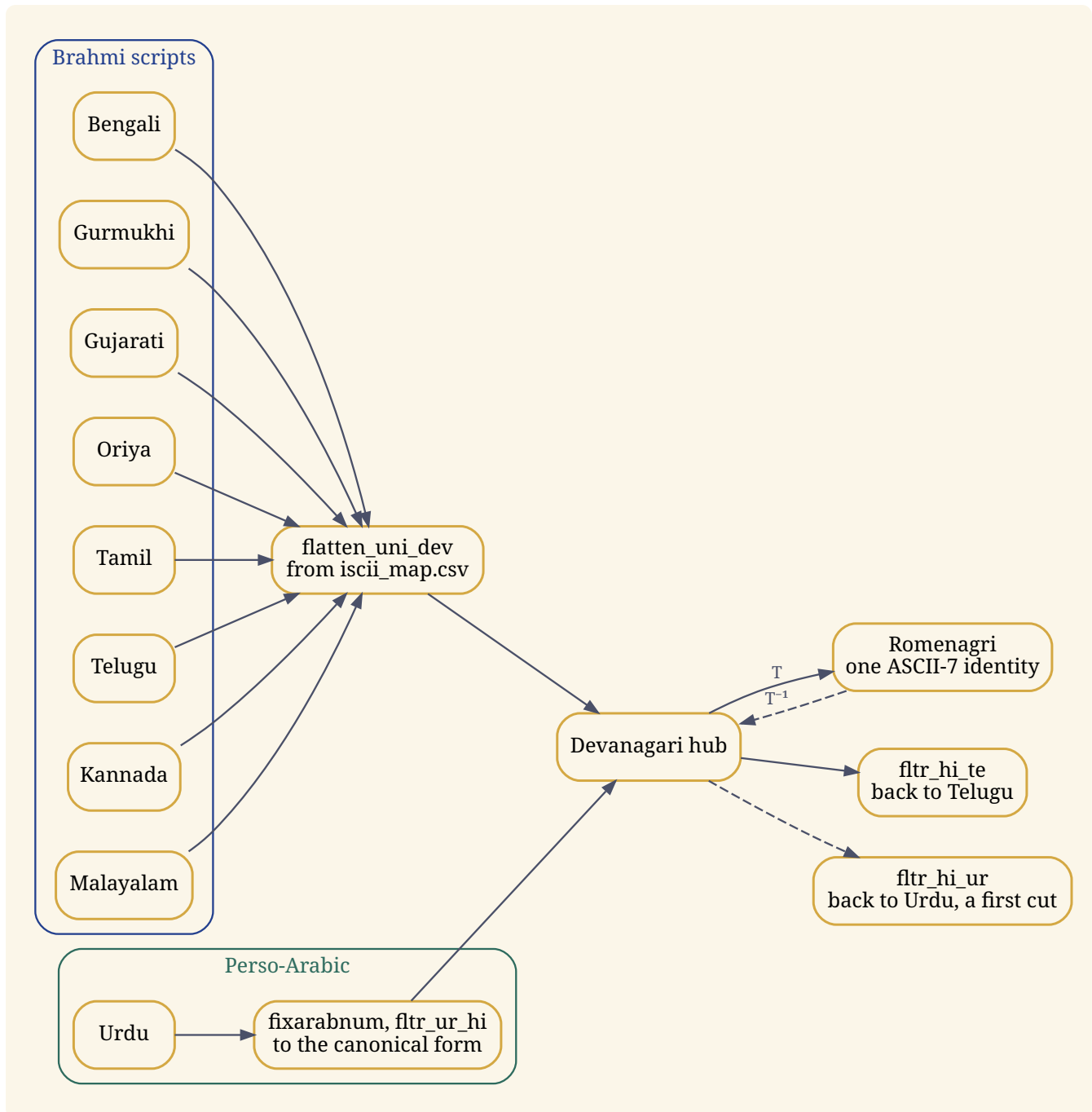


Figure 10. One identity across scripts: Brahmi scripts meet Devanagari in one hub; Perso-Arabic meets it at its written, canonical form.

This is the second pillar of linguistic equity at work. Because the script layer is independent of the language layer, any language can be written in any script, Hindi in Telugu letters or Telugu in Devanagari, and a program written either way is the same program to the compiler and the same names to the debugger.

Executed while this guide was built: `bash examples/hindawi.sh scripts`

1. The Brahmi hub: a word in Telugu or Bengali goes to Devanagari, to Romenagri, and back
 ప్రకృతి -> प्रकृति -> prak_ri_ti -> ప్రకృతి
 తెలుగు -> తెలుగు -> _taelugu -> తెలుగు
 গুণিত -> प्रकृति -> prak_ri_ti -> প্রকৃতি
 বাংলা -> বাংলা -> baa_mlaa -> বাংলা
2. Any language in any script
 Hindi written in Telugu script: हिंदी भाषा -> హిందీ భాషా
 Telugu written in Devanagari: తెలుగు భాష -> తెలుగు భాష
 Urdu written in Devanagari: ہندوی کتاب -> हन्दवी कताब
 the same, with zer and sukun: ہندوی کتاب -> हिन्दवी किताब
3. Measured on the corpora in this tree
 Hindi, words of letters and vowel signs: 984 distinct, 983 identical after Romenagri and back
 Telugu, through the hub and back: 1416 distinct, 1347 identical; of the 69 others, 69 carry the long-u sign U+0C42, which fltr_hi_te returns as U+0942
 Urdu: 1 of 794 words carry any zabar, zer, pesh, sukun, shadda or tanwin

Of the 984 distinct Hindi words made of letters and vowel signs, 983 come back identical; the exception is a malformed word in the corpus, with three u signs in a row, which comes back in its canonical form. Of the 1,416 Telugu words taken through the hub and back, 1,347 are identical, and every one of the other 69 carries the long-u sign ు, for which the reverse filter in this tree has no rule, so it returns the Devanagari sign: a one-line gap in `fltr_hi_te`, not in Romenagri. The author's own measurements in [project-ilm/romenagri](#) report 99.47 per cent of 5,807 words across the Hindi, Bengali and Telugu corpora exact. From the other side, over every string of up to four ASCII letters, his paper reports the 2003 to 2004 kernel as 98.68 per cent reversible or canonicalisable, with an irreducible floor of 1.31 per cent; the figure depends on the variant of the kernel measured, and his runbook says so.

The Perso-Arabic frontier

An abjad writes consonants and long vowels. The short vowels, zabar, zer and pesh, the absence of a vowel, sukun, and doubling, shadda, are marks that everyday Urdu leaves out: in the Urdu corpus in this tree, one word in 794 carries any of them. A written Urdu word therefore reaches the hub as what is written, its canonical form, and ہندوی arrives as हन्दवी, not हिंदवी. The mapping is exact and reversible to that form. What the script does not carry is which short vowel a speaker says.

With the marks written, the hub form is exact: ہندوی arrives as हिन्दवी, the author's own example in `hindawi_tashkil.txt`. So the difficulty is phonetic rather than a failure of the mapping. Which vowel a word carries has to come from knowledge of the language, its lexicon and its phonology, and that is precisely why ILM keeps the language axis apart from the script axis: the script layer maps what is written, exactly, and the language layer supplies what speakers know. The author estimates the words that need it at some 15 to 20 per cent.

Urdu programs already compile through the hub, as Chapter 21 shows. The reverse filter that renders hub text back into Urdu script, `fltr_hi_ur`, used by `urducc -r`, is a first cut in these trees; hardening the Perso-Arabic direction is the next step his seed release of Romenagri names. The Northwest Semitic abjads follow the same route to the Arabic hub: Hebrew final letter forms merge there, a documented many-to-one, and are reversible to the canonical form.

When is a transliteration reversible?

Let Σ be the source alphabet, the letters and marks of a script, and Γ the target alphabet, for example ASCII letters. A transliteration assigns each $\sigma \in \Sigma$ a non-empty target word $T(\sigma) \in \Gamma^+$ and extends to strings letter by letter:

$$T(\sigma_1\sigma_2\cdots\sigma_n) = T(\sigma_1)T(\sigma_2)\cdots T(\sigma_n).$$

It is reversible exactly when this extended map is injective, that is, when $T(s) = T(s')$ implies $s = s'$ for all strings $s, s' \in \Sigma^*$. Then there is an inverse with $T^{-1}(T(s)) = s$ for every string, which is the round-trip property a test can check. Injectivity holds if and only if T is one-to-one on single letters and the set of target words $C = T(\Sigma)$ is a *uniquely decodable code*.

Prefix codes. If no word of C is a prefix of another, every target string can be split into words from the left without looking ahead, so C is uniquely decodable.

Kraft and McMillan. If C is uniquely decodable over an alphabet of D symbols, with word lengths $\ell_1, \dots, \ell_m$, then

$$\sum_{i=1}^m D^{-\ell_i} \leq 1,$$

and whenever lengths satisfy this inequality, a prefix code with those lengths exists. A code whose sum exceeds 1 cannot be reversible, whatever else is done to it.

Sardinas and Patterson. For sets of words A and B write $A^{-1}B = \{w : aw \in B \text{ for some } a \in A\}$. Put $S_1 = C^{-1}C \setminus \{\varepsilon\}$ and $S_{k+1} = C^{-1}S_k \cup S_k^{-1}C$. The code is uniquely decodable if and only if no S_k contains a word of C . The sets are built from finitely many suffixes, so the test ends.

The example below runs both tests on four codes. The first is the mistake every naive scheme makes: `k` for `ک`, `h` for `ھ` and `kh` for `خ`. The string `kh` then reads both as `خ` and as `ک` followed by `ھ`, and the two tests catch it independently: the Kraft sum is $\frac{1}{2} + \frac{1}{2} + \frac{1}{4} = \frac{5}{4} > 1$, and the dangling suffix `h` is itself a codeword. The second is Romenagri's choice, as the output above shows: `ھ` carries the marker, `_h`, so the letter `h` never stands alone as a codeword and `kh` has exactly one reading.

Executed while this guide was built: `python3 examples/ud_check.py`

```
naive: k, h, kh      D=2  Kraft sum=5/4      (1.250)  uniquely decodable: False
Romenagri's choice: k, _h, kh  D=3  Kraft sum=5/9      (0.556)  uniquely decodable: True
not prefix-free: a, ab, bb  D=2  Kraft sum=1      (1.000)  uniquely decodable: True
prefix code: 0, 10, 110, 111  D=2  Kraft sum=1      (1.000)  uniquely decodable: True
```

```
"kh" under the naive code has 2 readings: k + h; kh
```

"khk" under the naive code has 2 readings: k + h + k; kh + k
"abbb" under a, ab, bb has 1 reading: ab + bb

The third code shows that a code can be uniquely decodable without being a prefix code; the price is lookahead, since **a** followed by **b** cannot be settled until the next symbols arrive. An abugida adds one more demand. KA with its inherent vowel, KA with a virama, and KA in a conjunct must all remain distinguishable in the target. Romenagri meets it by writing the inherent vowel (**ka**), by letting a consonant with no vowel after it stand for the virama (**k_ha** , **kx**), and by marking what would otherwise merge; and it keeps its whole alphabet inside the identifier alphabet, so that every transliterated name is at once a valid C identifier, a valid assembler symbol and a valid linker name. The domain of reversibility, whether all Unicode strings or only well-formed syllables, still has to be stated rather than assumed; the 67 entries above are the check this guide ran.

20. Pāṇini's grammar as a formal system

In plain words. Pāṇini's grammar of Sanskrit, written well over two thousand years ago, works like a computer program, and this chapter runs part of it.

The Aṣṭādhyāyī, “eight chapters”, is a generative grammar of Sanskrit, usually dated between the sixth and the fourth centuries BCE. It has close to four thousand sūtras, 3,959 in the usual count, arranged in eight *adhyāyas* of four *pādas* each and cited by chapter, quarter and number, so that 6.1.77 is the seventy-seventh sūtra of the first quarter of the sixth chapter. It works with auxiliary lists: the Śiva Sūtras, which order the sounds; the Dhātupāṭha, the list of verbal roots; and the Gaṇapāṭha, lists of words that rules refer to as classes. Around it grew a review tradition: Kātyāyana's *vārttikas* on individual rules (around the third century BCE), Patañjali's *Mahābhāṣya* (around the second century BCE), the *Kāśikā* (seventh century CE), and Bhaṭṭoji Dīkṣita's *Siddhāntakaumudī* (seventeenth century), which rearranged the rules by topic for learners.

What follows treats the grammar the way a computer scientist would: as data structures and a rewriting system. Every claim below can be checked by running the program at the end of the chapter:

The Śiva Sūtras: a sound inventory with tags

Fourteen short lines list the sounds of the language. Each line ends in an *it*, a marker that is not pronounced and not part of the inventory; it is a tag.

No.	Devanagari	Sounds (IAST)	Marker
1	अ इ उ ण्	a i u	ṇ
2	ऋ लृ क्	r ṛ	k
3	ए ओ ङ्	e o	ṅ
4	ऐ औ च्	ai au	c
5	ह य व र ट्	h y v r	ṭ
6	ल ण्	l	ṇ
7	ज म ङ ण न म्	ñ m ṇ ṇ n	m
8	झ भ ञ्	jh bh	ñ
9	घ ढ ध ष्	gh ḍh dh	ṣ
10	ज ब ग ड द श्	j b g ḍ d	ś
11	ख फ छ ठ थ च ट त व्	kh ph ch ṭh th c ṭ t	v
12	क प य्	k p	y
13	श ष स र्	ś ṣ s	r
14	ह ल्	h	l

Pratyāhāra: character classes by interval

Sūtra 1.1.71, *ādir antyena sahetā*, says that a first sound together with a later marker names every sound from the first up to that marker. So *ac* runs from *a* in line 1 to the marker *c* at the end of line 4 and names all the vowels; *hal* runs from *h* in line 5 to the final *l* and names all the consonants; *ik* is *i u ṛ ḷ*; *yaṇ* is *y v r l*. Each such abbreviation, a *pratyāhāra*, is exactly a character class defined by an interval over an ordered list, the same device as `[a-z]` in a regular expression, but over a list arranged so that the classes the grammar needs come out contiguous. The marker *ṇ* occurs twice, in lines 1 and 6, so *aṇ* is ambiguous between *a i u* and the longer run to line 6; the grammar knows from context which is meant. The sound *h* appears twice, in lines 5 and 14, so that it falls into the classes it must belong to.

How well the fourteen lines solve that arrangement problem has been studied formally: Kiparsky (1991) explained the arrangement by principles of economy, and Petersen (2004) analysed it mathematically, including why one sound has to be listed twice for every class the grammar needs to come out as an interval.

Kinds of sūtra

- *Samjñā*, a definition: 1.1.1 *vṛddhir ādaic* defines *vṛddhi* as *ā, ai, au*; 1.1.2 *adeṇ guṇaḥ* defines *guṇa* as *a, e, o*.
- *Paribhāṣā*, a meta-rule about how other rules are read and applied.
- *Vidhi*, an operation that changes a form.
- *Niyama*, a restriction on another rule.
- *Atideśa*, an extension that treats one thing as another.
- *Adhikāra*, a heading whose scope runs over the rules after it.

Compression and scope

A word stated in one sūtra continues into the following sūtras until something cancels it. This *anuvṛtti* lets a rule be written with only what is new in it; the rest is inherited from context, the way a variable declared in an enclosing block is visible inside it, or the way a default argument is supplied unless overridden.

Deciding which rule applies

Real grammars have rules that compete. The Aṣṭādhyāyī resolves conflicts with stated principles, and they are the part most worth studying.

- **General rule and exception.** An *apavāda*, the specific rule, blocks the *utsarga*, the general rule, where both could apply: the principle a pattern matcher follows when it prefers the more specific pattern.
- **1.4.2 *vipratishedhe param kāryam*.** In a conflict between rules of equal standing, the later rule in the text applies.

- **8.2.1 *pūrvatrāsiddham*.** The rules of the last three quarters of the eighth chapter, the *tripādī*, are treated as not having applied, as far as every earlier rule is concerned. It is an ordered pipeline: a late stage whose output the earlier stages are not allowed to see, as a compiler's later passes are invisible to its earlier ones.
- **1.1.56 *sthānivad ādeśo 'nalvidhau*.** A substitute behaves like what it replaced, except for rules about individual sounds: substitution through a shared interface.
- **1.3.2, 1.3.3 and 1.3.9.** Nasalised vowels and final consonants in teaching forms are markers, and markers are deleted: tags are stripped once they have done their work, as annotations are removed after parsing.

Four sandhi rules, computed

When words meet, their sounds change by rule. The program below builds the pratyāhāras from the table above by sūtra 1.1.71 and applies four rules of external sandhi to eight pairs.

- 6.1.101 *akaḥ savarṇe dīrghaḥ*: a simple vowel followed by a similar vowel becomes one long vowel.
- 6.1.77 *iko yaṇ aci*: a sound of class *ik* becomes the corresponding sound of class *yaṇ* before a vowel, *ac*. As a rewrite rule: $\{i, u, r, l\} \rightarrow \{y, v, r, l\} / \text{--- } ac$.
- 6.1.87 *ād guṇaḥ*: *a* or *ā* followed by a vowel of class *ik* gives the *guṇa* vowel.
- 6.1.88 *vrddhir eci*: *a* or *ā* followed by *e*, *o*, *ai* or *au* gives the *vrddhi* vowel.

File: `examples/panini_demo.py`

```
# panini_demo.py: the Śiva Sūtras, pratyāhāra formation (1.1.71) and three sandhi rules.
# © 1993-2026 Abhishek Choudhary. All rights reserved. AyeAI.
# SPDX-License-Identifier: GPL-3.0-or-later
# A teaching model in IAST. It is not tajziya's parser and makes no claim to cover the grammar.

SUTRAS = [ # (sounds, it-marker), the fourteen Maheshvara (Shiva) Sūtras
    (["a", "i", "u"], "ṇ"), (["ṛ", "ḷ"], "k"), (["e", "o"], "ṇ"), (["ai", "au"], "c"),
    (["h", "y", "v", "r"], "ṭ"), (["l"], "ṇ"), (["ñ", "m", "ṇ", "n"], "m"),
    (["jh", "bh"], "ṇ"), (["gh", "ḍh", "dh"], "ṣ"), (["j", "b", "g", "ḍ", "d"], "ś"),
    (["kh", "ph", "ch", "ṭh", "th", "c", "ṭ", "t"], "v"), (["k", "p"], "y"),
    (["ś", "ṣ", "s"], "i"), (["h"], "l"),
]

def pratyahara(first, marker, occurrence=1):
    """1.1.71 ādir antyena sahetā: a first sound with a final it-marker names every sound
    from the first up to the marker. Where a marker occurs twice (ṇ), say which one."""
    out, started, seen = [], False, 0
    for sounds, it in SUTRAS:
        for s in sounds:
            if s == first and not started:
                started = True
            if started:
                out.append(s)
            if started and it == marker:
                seen += 1
            if seen == occurrence:
                return list(dict.fromkeys(out)) # a sound listed twice (h) counts once
    raise ValueError(f"no pratyāhāra {first}{marker}")

AC = pratyahara("a", "c") # all vowels
HAL = pratyahara("h", "l") # all consonants
```

```

IK = pratyahara("i", "k")
YAN = pratyahara("y", "ṇ", 1) # the only ṇ after y is the second one in the list

LONG = {"a": "ā", "i": "ī", "u": "ū", "ṛ": "ṝ"}
SAVARNA = {k: k for k in LONG} | {v: k for k, v in LONG.items()}
YAN_OF = {"i": "y", "ī": "y", "u": "v", "ū": "v", "ṛ": "r", "ṝ": "r", "ḷ": "l"}
GUNA = {"i": "e", "ī": "e", "u": "o", "ū": "o", "ṛ": "ar", "ṝ": "ar"}
VRDDHI = {"e": "ai", "ai": "ai", "o": "au", "au": "au"}

def first_vowel(w):
    return w[:2] if w[:2] in ("ai", "au") else w[0]

def join(left, right):
    x, y = left[-1], first_vowel(right)
    if x in SAVARNA and y in SAVARNA and SAVARNA[x] == SAVARNA[y]: # 6.1.101 akaḥ savarṇe dīrghaḥ
        return left[:-1] + LONG[SAVARNA[x]] + right[len(y):], "6.1.101 akaḥ savarṇe dīrghaḥ"
    if x in YAN_OF and y in AC: # 6.1.77 iko yaṇ aci
        return left[:-1] + YAN_OF[x] + right, "6.1.77 iko yaṇ aci"
    if x in ("a", "ā") and y in VRDDHI: # 6.1.88 vṛddhir eci
        return left[:-1] + VRDDHI[y] + right[len(y):], "6.1.88 vṛddhir eci"
    if x in ("a", "ā") and y in GUNA: # 6.1.87 ād guṇaḥ
        return left[:-1] + GUNA[y] + right[len(y):], "6.1.87 ād guṇaḥ"
    return left + right, "no vowel sandhi"

if __name__ == "__main__":
    print("ac =", " ".join(AC))
    print("hal =", " ".join(HAL), f"({len(HAL)} consonants)")
    print("ik =", " ".join(IK), " yaṇ =", " ".join(YAN))
    print("aṇ (first ṇ) =", " ".join(pratyahara("a", "ṇ", 1)),
          " aṇ (second ṇ) =", " ".join(pratyahara("a", "ṇ", 2)))
    for a, b in [("dadhi", "atra"), ("madhu", "ari"), ("deva", "ālaya"), ("guru", "upadeśa"),
                  ("rāma", "iti"), ("mahā", "indra"), ("deva", "ṛṣi"), ("tava", "eva")]:
        w, rule = join(a, b)
        print(f"{a} + {b} -> {w:12s} by {rule}")

```

Executed while this guide was built: `python3 examples/panini_demo.py`

```

ac = a i u ṛ ḷ e o ai au
hal = h y v r l ñ m ṇ ṇ n jh bh gh ḍh dh j b g ḍ d kh ph ch ṭh th c ṭ t k p ś ṣ s (33 consonants)
ik = i u ṛ ḷ yaṇ = y v r l
aṇ (first ṇ) = a i u aṇ (second ṇ) = a i u ṛ ḷ e o ai au h y v r l
dadhi + atra -> dadhyatra by 6.1.77 iko yaṇ aci
madhu + ari -> madhvāri by 6.1.77 iko yaṇ aci
deva + ālaya -> devālaya by 6.1.101 akaḥ savarṇe dīrghaḥ
guru + upadeśa -> gurūpadeśa by 6.1.101 akaḥ savarṇe dīrghaḥ
rāma + iti -> rāmeti by 6.1.87 ād guṇaḥ
mahā + indra -> mahendra by 6.1.87 ād guṇaḥ
deva + ṛṣi -> devarṣi by 6.1.87 ād guṇaḥ
tava + eva -> tavaiva by 6.1.88 vṛddhir eci

```

The program tries the rules in an order chosen by hand. Pāṇini's system does not need a hand-chosen order: the meta-rules above decide it, and that is the difference between a script that produces the right answers for eight examples and a grammar that produces them for the language.

Pāṇini and formal language theory

A modern reader will recognise context-sensitive rewrite rules, ordered rule application, meta-rules for conflict, and a generative system that derives surface forms from roots and affixes. In 1967 Peter Ingerman proposed in *Communications of the ACM* that the notation used to define programming languages, Backus-Naur Form, be called Pāṇini-Backus Form, since Pāṇini's notation is equivalent in power. Pāṇinian derivation has been implemented in code, for example in vidyut-prakriya, whose sūtra text tajziya uses under the MIT licence. Tamil has its own grammatical tradition beginning with the Tolkāppiyam, and ILM's parsers are organised by language family so that such traditions sit side by side.

Questions to work through

1. Compute the pratyāhāras *jhal* and *yar* from the table by hand. Which sounds does each contain?
2. Why must *h* appear twice in the Śiva Sūtras?
3. Derive *madhu* + *ari* by hand. Which rule applies, and why does 6.1.101 not apply?
4. What would go wrong in a derivation if 8.2.1 were removed?
5. Compare 1.4.2 with the way a parser generator settles a shift-reduce conflict by precedence (Chapter 21).
6. Pāṇini's inventory lists sounds; Unicode's Devanagari block lists written signs. Which of the two is a phonology and which a script, and what does each leave out?

Part V: Working with the estate

27. Syllabus links, and our syllabus

In plain words. Links from your school or exam syllabus to the chapters that teach the same ideas, worked out on your own device, and a draft syllabus of our own.

A student's syllabus and this guide cover many of the same ideas under different names. This chapter links them without copying a word of any syllabus: the guide stores only the official address of each syllabus and its own list of keywords, and the matching happens on the reader's own device.

How the link works. Choose a board or exam; the page opens its official syllabus, from the publisher's own site. If the browser may read it directly, the page reads it; if the publisher does not allow that, which most do not, save the syllabus from the official page and drop the file on this page. Either way, the text is read in your browser, matched against the guide's own keywords, and every match becomes a link into the chapter that teaches it. Nothing is uploaded, stored or sent anywhere, and the syllabus remains the property of its publisher.

The boards and exams linked

Board or exam	Body	Levels	Region	Official page
GATE 2027, Computer Science and Information Technology	IIT Madras	Graduate	India	gate2027.iitm.ac.in
GATE 2027, Robotics and Automation	IIT Madras	Graduate	India	gate2027.iitm.ac.in
GATE 2027, Instrumentation Engineering	IIT Madras	Graduate	India	gate2027.iitm.ac.in
GATE 2027, General Aptitude	IIT Madras	Graduate	India	gate2027.iitm.ac.in
GATE 2027, every paper	IIT Madras	Graduate	India	gate2027.iitm.ac.in
CBSE, classes 9 to 12	Central Board of Secondary Education	Secondary, senior secondary	India	cbseacademic.nic.in
NCERT textbooks	National Council of Educational Research and Training	Classes 1 to 12	India	ncert.nic.in
ICSE, class 10	Council for the Indian School Certificate Examinations	Secondary	India	cisce.org

Board or exam	Body	Levels	Region	Official page
ISC, class 12	Council for the Indian School Certificate Examinations	Senior secondary	India	cisce.org
NIOS	National Institute of Open Schooling	Secondary, senior secondary	India	www.nios.ac.in
Cambridge IGCSE	Cambridge International Education	Secondary	International	www.cambridgeinternational.org
Cambridge International AS and A Level	Cambridge International Education	Senior secondary	International	www.cambridgeinternational.org
IB Diploma Programme	International Baccalaureate	Senior secondary	International	www.ibo.org
IB Middle Years Programme	International Baccalaureate	Middle, secondary	International	www.ibo.org
Advanced Placement courses	College Board	Senior secondary	United States	apcentral.collegeboard.org
Common Core State Standards	National Governors Association and CCSSO	Kindergarten to grade 12	United States	www.thcorestandards.org
AQA GCSE and A level	AQA	Secondary, senior secondary	United Kingdom	www.aqa.org.uk
Pearson Edexcel qualifications	Pearson	Secondary, senior secondary	United Kingdom	qualifications.pearson.com
OCR qualifications	OCR	Secondary, senior secondary	United Kingdom	www.ocr.org.uk
Australian Curriculum, version 9	ACARA	Foundation to year 10	Australia	v9.australiancurriculum.edu.au
Singapore national examinations	SEAB	Secondary, pre-university	Singapore	www.seab.gov.sg
Telangana Board of Secondary Education	Government of Telangana	Secondary	India, state board	bse.telangana.gov.in
Andhra Pradesh Board of Secondary Education	Government of Andhra Pradesh	Secondary	India, state board	bse.ap.gov.in
Maharashtra State Board	Government of Maharashtra	Secondary, higher secondary	India, state board	mahahsscboard.in

Board or exam	Body	Levels	Region	Official page
Karnataka School Examination and Assessment Board	Government of Karnataka	Secondary	India, state board	kseab.karnataka.gov.in
Tamil Nadu Directorate of Government Examinations	Government of Tamil Nadu	Secondary, higher secondary	India, state board	dge.tn.gov.in
Kerala SCERT	Government of Kerala	School	India, state board	scert.kerala.gov.in
West Bengal Board of Secondary Education	Government of West Bengal	Secondary	India, state board	wbse.wb.gov.in
Uttar Pradesh Madhyamik Shiksha Parishad	Government of Uttar Pradesh	Secondary, intermediate	India, state board	upmsp.edu.in
Board of Secondary Education, Rajasthan	Government of Rajasthan	Secondary, senior secondary	India, state board	rajeduboard.rajasthan.gov.in
Gujarat Secondary and Higher Secondary Education Board	Government of Gujarat	Secondary, higher secondary	India, state board	www.gseb.org
Punjab School Education Board	Government of Punjab	Secondary, senior secondary	India, state board	www.pseb.ac.in

The GATE papers' addresses were retrieved for this guide on 29 September 2026; the other addresses are the publishers' own sites, where the current syllabus is published. Each carries its review mark in the register:

Our syllabus, a draft for review

A standard syllabus built from this guide, level by level, with its own outcomes, chapters and laboratory work. It is a draft for the author's review, and each module carries its review mark in the register at the end of the guide.

Foundation, classes 6 to 8

Module	Title	Hours	Outcomes	In this guide	Laboratory
F1	Numbers inside the machine	6	Count in binary and convert to decimal; Explain why a computer needs only two symbols	Chapter 17	

Module	Title	Hours	Outcomes	In this guide	Laboratory
F2	Letters, scripts and your language	6	Show that every letter is stored as a number; Write a word in two scripts and say what stays the same	Chapter 18, Chapter 19	
F3	The sky and the calendar	4	Read sunrise, moonrise and the tithi for a day; Explain why calendars differ	Chapter 10	chakra, jyotish
F4	Making with a cycler, safely	4	Make a short story or picture with a cycler; Say what you checked before keeping the result	Chapter 24	genie

Secondary, classes 9 and 10

Module	Title	Hours	Outcomes	In this guide	Laboratory
S1	Binary, logic and circuits	10	Build truth tables for basic gates; Relate a gate to transistors	Chapter 17, Chapter 22	
S2	Encoding and reversible transliteration	8	Explain UTF-8; Run a Romenagri round trip and judge whether it is exact	Chapter 18, Chapter 19	hindawi
S3	Programming in your own language	10	Compile and run a Hindi program; Find the same names in the debugger	Chapter 21	hindawi, urdu
S4	Using AI and checking it	6	Use any AI on a task and record each step; Tell an AI's claim from a verified result	Chapter 24, Chapter 8	alam

Senior secondary, classes 11 and 12

Module	Title	Hours	Outcomes	In this guide	Laboratory
H1	Grammar as a formal system	10	Write a regular expression and a small grammar; Read a Pāṇinian rule as a rewrite rule	Chapter 20, Chapter 21	tajziya
H2	From source code to machine	12	Trace a program from source through compiler, assembler and linker; Read symbols in an ELF file	Chapter 21, Chapter 22	hindawi
H3	Open source, provenance and contributing	6	Make a commit, open a pull request and pass a gate; Explain a hash, a manifest and a DOI	Chapter 25	rahnuma
H4	A project seeded from FAKIR	15	Choose a domain, a language and a layer and state a gate; Deliver the work with its evidence	Chapter 29	fakir

Undergraduate and GATE

Module	Title	Hours	Outcomes	In this guide	Laboratory
U1	Theory of computation and compilers	30	Automata, grammars and parsing; Build and debug a compiler pipeline	Chapter 20, Chapter 21, Chapter 26	hindawi , panini_by_claude
U2	Digital logic and architecture	30	From gates to a processor; Ternary logic and its trade-offs	Chapter 22, Chapter 26	pratik , zamin
U3	Quantum and physical computing	20	Qubits, gates and QAOA; Oscillator networks and Ising problems	Chapter 23	paniniq

Postgraduate and research

Module	Title	Hours	Outcomes	In this guide	Laboratory
P1	Research on the open questions	60	Choose an open question from Appendix C or D; Publish a measured, reproducible result	Appendix C, Appendix D, Appendix F	paniniq , tajziya

Professional

Module	Title	Hours	Outcomes	In this guide	Laboratory
Q1	Adopting a component	12	Run a component's gate; Integrate it with its contract and provenance intact	Chapter 15, Chapter 25, Appendix E	dhancha , humanesque

© 1993–2026 Abhishek Choudhary. All rights reserved. AyeAI. Text CC BY-SA 4.0; scripts GPL-3.0-or-later. Version 2.0.0, 3 October 2026.